[Submitted on 28 Nov 2023 (v1), last revised 17 Jun 2024 (this version, v2)]

Large Language Models Suffer From Their Own Output: An Analysis of the Self-Consuming Training Loop

Martin Briesch[1], **Dominik Sobania**[1], **Franz Rothlauf**[1]
[1]Johannes Gutenberg University, Mainz, Germany,
Correspondence: briesch@uni-mainz.de

arXiv:2311.16822v2 [cs.LG] 17 Jun 2024

Abstract

Large Language Models (LLM) are already widely used to generate content for a variety of online platforms. As we are not able to safely distinguish LLM-generated content from human-produced content, LLM-generated content is used to train the next generation of LLMs, giving rise to a self-consuming training loop. From the image generation domain we know that such a self-consuming training loop reduces both quality and diversity of images finally ending in a model collapse. However, it is unclear whether this alarming effect can also be observed for LLMs. Therefore, we present the first study investigating the self-consuming training loop for LLMs. Further, we propose a novel method based on logic expressions that allows us to unambiguously verify the correctness of LLM-generated content, which is difficult for natural language text. We find that the self-consuming training loop produces correct outputs, however, the output declines in its diversity depending on the proportion of the used generated data. Fresh data can slow down this decline, but not stop it. Given these concerning results, we encourage researchers to study methods to negate this process.

1 Introduction

Transformer-based language models have received much attention in the machine learning community in recent years. Especially large language models (LLM) trained on massive amounts of data from the internet became state of the art in many benchmarks (Brown et al., 2020) and specialized conversational LLM applications like ChatGPT[1] have a massive influence on society already. LLMs can be used for multiple tasks ranging from code generation (Chen et al., 2021; Sobania et al., 2022; Fan et al., 2023) and automated program repair (Sobania et al., 2023) to text summarization (Yang et al., 2023) and teaching assistance (Baidoo-Anu and Ansah, 2023).

Due to their convincing generated outputs, LLMs can be used to generate a large amount of content that is posted online to coding platforms like GitHub and Stackoverflow[2], social media platforms like Reddit[3] and other platforms on the internet. Even academic writing is already being influenced by LLM outputs (Geng and Trotta, 2024; Liang et al., 2024). Such LLM-generated text is often hard to distinguish from human-generated content (Sadasivan et al., 2023) and in turn might unwillingly be used to train the next generation of LLMs. Even paying for human-generated content might not be an option in the future, as workers at paid services like Amazon's Mechanical Turk also use LLMs to produce content (Veselovsky et al., 2023). Consequently, a self-consuming training loop emerges in which future models are trained repeatedly on LLM-generated data from previous generations. This process was first observed for generative models in the image domain (Martínez et al., 2023; Alemohammad et al., 2023; Shumailov et al., 2023; Bertrand et al., 2023). These studies found that this self-consuming training loop leads to a decline in quality and diversity of generated images, ultimately resulting in a so called model collapse. However, it is unclear what happens with LLMs that are trained in such a self-consuming training loop from scratch, as usually done in real-world applications like ChatGPT.

Therefore, we present the first study analyzing the behavior of LLMs trained over many generations in a self-consuming training loop. We conduct experiments on a GPT-style model in different settings and measure both quality and diversity of samples from the trained model at each generation

[1]https://openai.com/blog/chatgpt

[2]https://www.microsoft.com/en-us/Investor/events/FY-2023/Morgan-Stanley-TMT-Conference

[3]https://www.vice.com/en/article/jg5qy8/reddit-moderators-brace-for-a-chatgpt-spam-apocalypse

in the self-consuming training loop. The settings differ in the way a dataset is created for each generation (so called data cycles) as well as the proportion of original *real* and LLM-generated *synthetic* data samples. To better analyze the behavior of the trained models we conduct our experiments on a dataset consisting of logic expressions. In contrast to natural language, this logic expressions can be evaluated unambiguously. This allows us to analytically and accurately measure correctness and diversity of the generated samples.

We find that repeatedly training new models with synthetic data from previous models initially improves quality. However, diversity degenerates and the learned distribution inevitably collapses to a single point. In extreme cases this happens already after less than 10 generations. Additionally, we find that the speed at which diversity degenerates depends on the data cycle as well as on the proportion of real and synthetic data. Fresh real data added during the data cycle slows down but can not negate the effects of a self-consuming training loop.

In summary, our main contributions are as follows:

- The first comprehensive empirical study of the self-consuming training loop for LLMs trained from scratch,
- A novel method to unambiguously evaluate quality/correctness and diversity of LLM-generated outputs,
- An in-depth analysis and discussion of the effects and implications of this self-consuming training loop for LLMs.

Following this introduction, Sect. 2 describes the self-consuming training loop and gives an overview of related work. In Sect. 3, we describe our experimental setting. Section 4 presents the experimental results, followed by a discussion in Section 5. Section 6 concludes the paper.

2 The Self-Consuming Training Loop

Current generations of LLMs are usually trained on large amounts of unstructured data like text and code gathered from the Internet (Brown et al., 2020). Due to their generative nature and capacity those models can in turn be used to generate new *synthetic* data, often indistinguishable from the original *real* data (Sadasivan et al., 2023). This synthetic data ends up back on the internet and thus in the next large dataset, which is used to train the next generation of LLMs. These LLMs in turn produce new content and data, setting in motion a repetitive cycle in which new generations of models are trained each time with a higher proportion of synthetic data from previous generations. We call this process a self-consuming training loop, depicted in Figure 1.

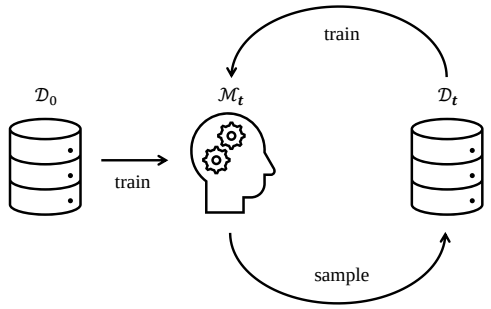

Figure 1: Self-consuming training loop: In the first generation $t = 1$ a model $\mathcal{M}_t$ is initially trained on a real dataset $\mathcal{D}_0$. From this model $\mathcal{M}_t$ a sample $\mathcal{S}_t$ is drawn to build a new dataset $\mathcal{D}_t$. The new dataset $\mathcal{D}_t$ is in turn used to train a new model $\mathcal{M}_{t+1}$ from scratch for the next generation $t + 1$. This process is repeated iteratively until the maximum number of generations T is reached.

Algorithm 1 Self-Consuming Training Loop

Input: number of generations T, data cycle
Initialize: Sample $\mathcal{D}_0$ from P_X
for $t = 1$ **to** $t = T$ **do**
  Train Model $\mathcal{M}_t$ with $\mathcal{D}_{t-1}$
  Sample $\mathcal{S}_t$ from $\mathcal{M}_t$
  Create $\mathcal{D}_t$ from $\mathcal{D}_0$ and $\mathcal{S}_{1...t}$ according to data cycle
end for

More specifically, consider a dataset $\mathcal{D}_0$ consisting of real data points $x \in X$ sampled from the original distribution P_X. A generative model $\mathcal{M}_t$ is trained on this original dataset $\mathcal{D}_0$ until the data is sufficiently fitted, producing the first generation $t = 1$ of generative models. In a self-consuming training loop, a new set of m synthetic data points $\mathcal{S}_t$ is now sampled from the previous generation of generative models $\mathcal{M}_t$. The next generation of generative models $\mathcal{M}_{t+1}$ is then trained from scratch on the new dataset $\mathcal{D}_t$. This self-consuming training loop is repeated until the maximum number of generations $t = T$ is reached. Algorithm 1 presents

a pseudo-code describing this self-consuming training loop.

Related Work Current research has analyzed the self-consuming training loop mostly in the context of image generating models. Martínez et al. (2023) first studied the self-consuming training loop for diffusion models and find that the self-consuming training loop leads to a collapse in diversity of the generated images. Other work have given theoretical frameworks for this phenomenon and look at more complex data cycles in the context of image generation (Alemohammad et al., 2023; Shumailov et al., 2023; Bertrand et al., 2023). Both Alemohammad et al. (2023) and Bertrand et al. (2023) find that fresh real data can lead to stability within the self-consuming training loop.

Shumailov et al. (2023) observe a degeneration of diversity for Variational Autoencoders and Gaussian Mixture Models if some of the output is used again as input. Additionally, the authors perform experiments for iteratively fine-tuning LLMs in a self-consuming way and observe a degradation in quality. They find that degradation is less strong than in the imagine generation context and that keeping some original data helps mitigating this phenomenon. Other contemporary work also analyzed repeated fine-tuning of LLMs in a self-consuming way and observed a decrease in lexical diversity (Guo et al., 2023).

However, new generations of LLMs are usually trained from scratch with web scraped datasets while fine-tuning is mainly done with curated datasets. So the analysis of the self-consuming training loop when training from scratch is of pressing concern. To the best of our knowledge, we are the first to study the behavior of LLMs trained in a self-consuming loop from scratch. Furthermore, self-consuming loops in LLMs have not yet been analyzed with a method that allows to unambiguously evaluate the quality and diversity of generated model output.

3 Experimental Setup

We present our logic expression dataset, which is the foundation for the verification of the language model's output. Furthermore, we explain the data cycles we analyze in our experiments and describe the used model architecture in detail.

3.1 Verification with a Logic Expression Dataset

LLMs usually generate natural language texts and their performance is typically measured by using similarity metrics like the BLEU score (Papineni et al., 2002), ROUGE score (Lin, 2004) and BERT score (Zhang et al., 2019) as well as perplexity (Jurafsky and Martin, 2009). However, those metrics rely on measuring similarity of outputs with expected reference data, which can only serve as a proxy for quality of a language model (Callison-Burch et al., 2006; Gehrmann et al., 2023). Consequently, we propose using a dataset consisting of logic expressions. Those expressions can be represented as a sequence making them a good fit for language modeling. In contrast to natural language text, we can easily and systematically evaluate the quality of logic expressions by verifying their correctness. An expression is defined to be syntactically correct, if it can be parsed without an error. The semantic correctness can be evaluated if an expression is either `True` or `False`. If the original dataset only consists of `True` expressions, then a high-quality trained model should also only generate `True` expressions.

We build a logic expression as a tree in a recursive way specified in the function `GenLogicExpr(`d`)`, where d is the desired depth of a logic expression tree. If the desired tree depth is reached when calling the function, a random Boolean is returned (either `True` or `False`). Otherwise, a logic operator (`not`,`and`,`or`) is selected and the function is called again recursively with $d-1$. To build the entire original dataset we use this recursive function to sample new random logic expression trees with a random depth between d_{min} and d_{max} until we have a dataset with m unique expressions that evaluate in a `True` Boolean expression. The complexity of the dataset can be easily controlled by adjusting d_{max}. For our experiments we chose a initial dataset size of $m = 10,000$ with expressions of minimum depth $d_{min} = 1$ and maximum depth $d_{max} = 5$. Algorithm 2 in Appendix A describes the generation of our logic expression dataset in pseudo-code.

The resulting logic expression trees can then be saved as strings, e.g., `not ( True and False )`. This allows us to evaluate them in Python using the `eval()` function to test whether they are correct or trigger an error and whether they evaluate to a `True` or `False` Boolean. Additionally, we can en-